

Relatorio de Vulnerabilidades

Sistema Integrado de Atendimento e Execucao de Servicos - Oficina Mecanica

Data: 2026-08-07

Ferramentas: bandit 1.9.4 (SAST) e pip-audit 2.10.1 (SCA)

Ambiente: Python 3.12.8

Como reproduzir

```
uv run bandit -r app scripts
uv run pip-audit
```

Resultados

SAST - bandit (analise estatica do codigo)

Total lines of code: 3320

Total issues (by severity): Undefined: 0, Low: 0, Medium: 0, High: 0

Nenhum achado em app/ e scripts/.

SCA - pip-audit (dependencias)

No known vulnerabilities found

Nenhuma vulnerabilidade conhecida nas dependencias instaladas (producao + dev).

Cobertura por categoria do OWASP Top 10 (2021)

Como os scanners nao encontraram achados, esta secao documenta como cada categoria do OWASP Top 10 e tratada pela arquitetura atual - e isso que embasa a confianca no resultado "limpo" acima, e tambem onde ficam registradas as lacunas conhecidas.

A01 - Broken Access Control [Mitigado]

RBAC por perfil (ADMIN/ATENDENTE/MECANICO) via dependency require_roles, aplicado por rota/roteador conforme a tabela de permissoes do desafio; perfil embutido como claim no JWT, nao em dado que o cliente possa manipular sem invalidar a assinatura.

A02 - Cryptographic Failures [Mitigado]

Senha de usuario nunca fica em texto puro (hash bcrypt); JWT assinado com HS256 via chave em variavel de ambiente (JWT_SECRET_KEY); valores monetarios usam Decimal, nunca float. Corrigido nesta rodada: o default de desenvolvimento de jwt_secret_key tinha 23 bytes, abaixo do minimo de 32 recomendado pela RFC 7518 par. 3.2 para HS256 - alertado pelo proprio PyJWT (InsecureKeyLengthWarning) durante a suite de testes. Ajustado para 42 bytes (continua um placeholder, sempre sobrescrito por env var em producao).

A03 - Injection [Mitigado]

Toda persistencia via SQLAlchemy ORM parametrizado - nenhuma concatenacao de SQL bruta no projeto. Entradas validadas na borda por schemas Pydantic antes de chegar a aplicacao; Documento/Placa/Money validam formato na propria construcao do Value Object.

A04 - Insecure Design [Mitigado]

Clean Architecture com regra de dependencia unica (dominio nao conhece framework); invariantes de negocio garantidas por Value Objects e pela maquina de estados explicita de OrdemServico (transicao invalida e rejeitada no dominio, nao checada por fora).

A05 - Security Misconfiguration [Mitigado]

Configuracao via pydantic-settings + .env (fora do controle de versao, ver .gitignore); excecoes de dominio

sao mapeadas para respostas HTTP genericas pelos exception handlers - nenhum stack trace ou detalhe interno e exposto ao cliente.

A06 - Vulnerable and Outdated Components [Verificado]

pip-audit sem vulnerabilidades conhecidas na data deste relatorio; uv.lock fixa versoes exatas. Precisa ser reexecutado periodicamente, ja que novas CVEs sao publicadas independentemente de mudanca no codigo.

A07 - Identification and Authentication Failures [Mitigado]

Login por JWT com expiracao (JWT_EXPIRACAO_MINUTOS); verificacao de senha sempre executa o bcrypt.checkpw contra um hash constante quando o e-mail nao existe, para nao vazar por tempo de resposta se um e-mail esta cadastrado; usuario inativo e bloqueado no login mesmo com senha correta.

A08 - Software and Data Integrity Failures [Mitigado]

Nenhum uso de pickle/eval/deserializacao insegura sobre dado nao confiavel; uv.lock garante builds reprodutíveis (mesmas versoes/hashees).

A09 - Security Logging and Monitoring Failures [Lacuna conhecida]

Nao ha logging estruturado de eventos de seguranga (tentativas de login falhas, mudancas de permissao). Fora do escopo do MVP, registrado aqui como melhoria futura.

A10 - Server-Side Request Forgery (SSRF) [Nao aplicavel]

A API nao faz requisicoes HTTP de saida a partir de entrada do usuario.

Achados corrigidos nesta rodada

- app/shared/settings.py: default de jwt_secret_key alongado de 23 para 42 bytes (RFC 7518 par. 3.2), eliminando o InsecureKeyLengthWarning observado na suite de testes.

Limitacoes desta analise

- bandit/pip-audit sao ferramentas automatizadas - nao substituem revisao manual nem pentest.
- pip-audit audita o ambiente instalado no momento da execucao; deve ser reexecutado a cada atualizacao de dependencia e periodicamente mesmo sem mudanca de codigo.